

Capability layer (Claude — and the same idea in any AI)

Source course: Associate · Module 1 · Claude Platform & Model Foundations · Capability Layer

Related note: [core-entry-points.md](#) (where you work)

Goal: decide *what* extends plain text generation so work is consistent, verified, and continuous.

Prereqs: understand Chat vs Project (entry points). No API coding required.

One-sentence definition

The **capability layer** is the set of product features that change *what the model can do* inside an entry point: run a fixed procedure (**Skills**), compute with real code (**Code Execution**), and carry facts across sessions (**Memory**). **Projects** supply workstream context; together they form a four-layer stack.

Why this exists

Default LLM output is fluent, not reliable:

Need	Default text alone	Capability that fixes it
Same procedure every time	You re-prompt; format drifts	Skill (or equivalent reusable procedure)
Numbers must be true	Model may invent plausible math	Code Execution (run the calculation)
Facts without re-typing	You re-paste every session	Memory + Project knowledge
Stable workstream background	Cold start each chat	Project instructions + knowledge

High-level map (four-layer model)

Course model — layers are **independent**; mix by task:

```
| PROJECT – context |
| standing instructions + knowledge base for this workstream |
```

- | SKILL – procedure (how this task type is done)
- | CODE EXECUTION – verify / compute (must be correct)
- | MEMORY – continuity (facts across sessions)

Layer	Question it answers
Projects	What background and standing rules apply to this workstream?
Skills	How should this task type be executed, every time?
Code Execution	When must the result be correct, not merely plausible?
Memory	What should carry forward without re-entry?

Fundamental unit: one *capability gap* (variance, computation risk, or continuity).

Mechanism: you attach the layer that closes that gap; a one-off question may need none; a monthly analytical workflow may use all four.

Entry points vs capability layer:

- Entry points = **where** you work (Chat, Project, Artifact, Research)
- Capability layer = **what** runs inside that place

Skills

What

A **Skill** is a reusable procedure package: instructions (often Markdown), optional scripts, and resources that Claude loads when the task matches. Official help: folders of instructions, scripts, and resources loaded dynamically for specialized tasks.

Mechanism

1. You enable built-in Skills and/or add custom Skills (account **settings**).
2. Skills live at **account level**, not inside one Project.
3. Claude **invokes automatically** when relevant — inside or outside a Project.
4. A Project still only configures **instructions + knowledge**; a Skill configures the **procedure** for a task type wherever it appears.

Anthropic ships built-ins for common pro work: create/edit/analyze **Excel, Word, PowerPoint, PDF**. Custom Skills cover your org's workflows.

Platform/API angle (same idea): Skills often run with a **code execution** environment; progressive disclosure loads only what the task needs ([Anthropic engineering on Agent Skills](#)).

Two hard rules (course)

1. **Reduce variance $\neq$ eliminate variance.** Same report Skill $\rightarrow$ same structure, different prose. **Human review stays.**
2. **Trust evaluation before enable.** A Skill can use whatever tools/files you expose in the session. Prefer Anthropic-provided and org-approved Skills; review third-party source + permissions. (Course points Module 6 for full evaluation.)

Use when

- Same multi-step process weekly/monthly
- Output format must match a template (deck, spreadsheet, PDF)
- You want the procedure available in **every** chat, not only one Project

Failure modes

Symptom	Fix
Skill never fires	Description/trigger mismatch; restate the task in Skill language
Wrong Skill fires	Narrow Skill scope; clearer task wording
Blind trust in output	Keep review step; variance remains
Shadow Skill from random internet	Do not enable; write your own or use org-approved

Not Claude-only

Elsewhere	Same job
Grok / Cursor / Claude Code <code>SKILL.md</code>	Auto-invoked procedure packs
Custom GPTs + actions	Reusable specialist behavior
Slash commands / macros	Explicit procedure invoke
Internal runbooks as tools	Procedure encoded outside freeform chat

Code Execution

What

Code Execution lets Claude **run code** (not only describe it) so math, transforms, and file generation are produced by a runtime, not by linguistic guesswork.

Why first principles

LLMs sample plausible next tokens. Arithmetic and multi-step transforms are where “sounds right” diverges from “is right.” Execution closes that gap: the model writes code → runtime returns the real result → you (or the report Skill) use that result.

Use when

- Totals, rates, date math, portfolio filters, unit conversions
- Spreadsheet transforms that must match formulas
- “Verify every numeric figure” in high-stakes reports

Failure modes

Symptom	Fix
Trusting a number in prose without a compute step	Force Code Execution / calculator / spreadsheet formula
Wrong code, right confidence	Review code or spot-check inputs/outputs
Using code for pure writing	Prefer Skill + Project; do not over-tool

Not Claude-only

ChatGPT Advanced Data Analysis / code interpreter, local REPL agents, notebook kernels, spreadsheet formulas — same principle: **verify outside the language model’s head.**

Memory

What

Memory stores work-relevant facts across sessions so you stop re-entering them. Typical contents: role context, output preferences, frequent collaborator names, standing constraints that apply across many chats.

Curation (course — non-negotiable)

Memory that was true last quarter can **mislead** now. Active users should:

1. Review at least **monthly**
2. Delete or update stale entries
3. Keep only what **genuinely recurs**

Project-scoped Memory

Separate Projects → separate Memory contexts. Client A facts should not appear in Client B. Same rule as Project hygiene: **one workstream per Project** when confidentiality or role differs.

Incognito

Incognito keeps a session out of Memory and chat history (standalone chats **outside** Projects). Use for sensitive or throwaway confidential inputs. It does **not** override org data-retention policy.

Import from other AIs

Course (as of **June 2026**): importing memories from other platforms is **experimental**. Documented for Free, Pro, Max, Team (**not Enterprise**). If auto-import is missing, add key facts **manually in Memory settings** — do **not** dump them only into Project knowledge as a substitute for Memory (different jobs: Memory = personal continuity; Project knowledge = workstream corpus).

Memory vs Project knowledge vs Skill

Store	Job	Lifetime / scope
Memory	Who you are / prefs / cross-cutting facts	Across sessions; project-scoped when inside Projects
Project knowledge	Docs and corpus for one workstream	That Project only
Standing instructions	How to behave in that Project	That Project only
Skill	How to run a task type	Account-wide, auto-invoked

Failure modes

Symptom	Fix
Wrong client details surface	Split Projects; audit Memory
Stale role (“still at old company”)	Monthly Memory review
Secrets in Memory	Delete; use Incognito for one-offs; follow org policy

Worked scenario (course) — monthly regulatory report

Job: each month, take regulatory updates → map to portfolio → summarize implications → format to a fixed template. High stakes, repeatable structure.

Months 1-2 (Chat only)

- Re-upload docs, re-paste portfolio, re-type format every time
- Manual verification of every number
- ~65 min/session; human caught 2 errors then 1

Month 3+ (full capability layer)

Piece	Layer
Portfolio context + format rules	Project instructions + knowledge
Prior reports as examples	Knowledge base
Report output procedure/template	Skill
Numeric calculations	Code Execution
(Optional) analyst role / prefs	Memory

Result (illustrative course numbers): ~**30 min**/session; verification still ran; **no errors** months 3-8.

Design questions she asked first

Question	Layer
Which parts are the same every time?	Standing instructions + Skill
Which reference material recurs?	Knowledge base
Which outputs must be computed correctly?	Code Execution
Which context should carry without re-entry?	Memory

More examples (beyond the course)

Example A — Weekly metrics pack for a product team

Layer	Content
Project	Product glossary, OKR doc, chart style rules
Skill	“Weekly metrics pack” → fixed slides/sections

Layer	Content
Code Execution	Growth rates, cohort math from raw CSV
Memory	“Prefer tables before narrative; no emoji”

Example B — Learning Associate Module 1 (you)

Layer	Content
Project	Course notes (this vault’s AI/*.md), quiz mistakes
Skill	Optional: “tutor quiz then correct” procedure
Code Execution	Rarely (unless practicing data tasks)
Memory	“Teaching style: first principles, frugal, 3-pass eval”

Example C — Contract redlines (checkpoint practice)

Map each component to a layer (practice set — not the live quiz cards):

Component	Layer
Always flag unlimited liability in red; never invent clause numbers	Standing instructions
Playbook PDF + preferred clause library	Knowledge base
Standard redline procedure (order of review, comment style)	Skill
Compute remaining contract term / fee pro-rate from dates	Code Execution
“Output: summary table then clause list” preference	Standing instructions or Memory if cross-project

Official checkpoint in the course UI may use different five cards; use the **design questions** table, not memorized labels, when unsure.

Example D — Research synthesis that still needs compute

1. **Research** (entry point) for multi-source market scan
2. **Code Execution** for TAM math from cited figures
3. **Artifact** for the deliverable
4. Save stable sources into **Project knowledge** for next quarter

Decision checklist (30 seconds)

1. Same steps every time? → **Skill** (and/or standing instructions)

2. Same documents every time? → **Project knowledge**
3. Numbers or transforms must be true? → **Code Execution**
4. Facts about *me/my role* across work? → **Memory**
5. Facts about *this client/workstream* only? → **Project**, not global Memory dump
6. Sensitive one-off? → **Incognito** (outside Projects)
7. Still re-pasting last month's brief? → missing **Project** or **Skill**

Checkpoint self-test

Without looking up:

1. Name the four layers and one sentence each.
2. Skills live in the Project — true/false? (**False** — account level.)
3. Skills remove the need for human review — true/false? (**False.**)
4. Why Code Execution for totals?
5. What is the monthly Memory habit?
6. Incognito: does it override org retention? (**No.**)

Answers are in the sections above.

Official Claude / Anthropic references

Topic	URL
What are skills? (Help Center)	https://support.claude.com/en/articles/12512176-what-are-skills
Agent Skills overview (Platform docs)	https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview
Equipping agents with Agent Skills (Engineering)	https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills
Complete Guide to Building Skills for Claude (PDF)	https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf
Code execution tool (API)	https://platform.claude.com/docs/en/agents-and-tools/tool-use/code-execution-tool
Chat search and memory	https://support.claude.com/en/articles/11817273-use-claude-s-chat-search-and-memory-to-build-on-previous-context
Effective context engineering for AI agents	https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
Projects (context layer)	https://support.claude.com/en/articles/9517075-what-are-projects
RAG for Projects	https://support.claude.com/en/articles/11473015-retrieval-augmented-generation-rag-for-projects

Gold-mine blogs (outside Claude product docs)

First-principles writing that generalizes beyond Claude UI labels. Prefer these when product screens change but the *problem* does not.

Source	Why it is gold
Anthropic — Effective context engineering for AI agents	Official engineering, but conceptual : context engineering vs prompt engineering — the right mental model for Memory + Projects + Skills
Anthropic — Equipping agents with Agent Skills	Why Skills + optional code beat stuffing everything into one system prompt
Lee Han Chung — Claude Agent Skills first-principles deep dive	Architecture: progressive disclosure, SKILL.md, scripts vs prompt bulk
Firecrawl — Context engineering vs prompt engineering	Clear TL;DR: prompt = inside the window; context eng = <i>what</i> fills the window (memory, tools, RAG, history)
Addy Osmani — How to write a good spec for AI agents	Specs, subagents, RAG — production agent discipline
Addy Osmani — AI coding workflow 2026	Durable workflows; Skills as packaged procedures
Simon Willison — many agent/tool essays	Trusted practitioner on tools, evaluation, and not trusting model math blindly
Miguel Fierro — Prompt vs Skill vs Agent	Ladder: prompt → skill when it repeats → agent when multi-step + tools
HackerNoon — Stop prompting. start engineering (15 principles)	External memory, state outside the agent, production failure modes

Source note

Spine from **Claude Associate · Module 1 · Capability Layer** (Skills & Code Execution, Memory, regulatory scenario, checkpoint idea). Product details cross-checked against Anthropic Help Center / platform docs where linked. Examples A-D and gold-mine table are operator expansions. UI plan limits can change — re-fetch official docs when studying.